

ИТОГОВЫЙ ПРОЕКТ · АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ

Планировщик маршрута курьера

Выбор заказов и порядка их доставки при ограничении смены и дедлайнах: алгоритм Дейкстры + динамическое программирование по подмножествам

C# · .NET 10 · 42 теста xUnit

Продолжение задачи KT7 «Маршрут дрона с ограниченным зарядом»

ЗАДАЧА

Больше ценных заказов — и назад до конца смены

Вход

- **Дорожная сеть** — ориентированный граф, вес дуги — минуты проезда (≥ 0), есть односторонние улицы
- **Склад и смена** T минут
- **Заказы:** адрес, ценность, дедлайн (необязательный), время передачи

Выход

- Какие заказы везти и в каком порядке, время прибытия, путь по улицам
- Недоставленные заказы — с причиной

Критерий

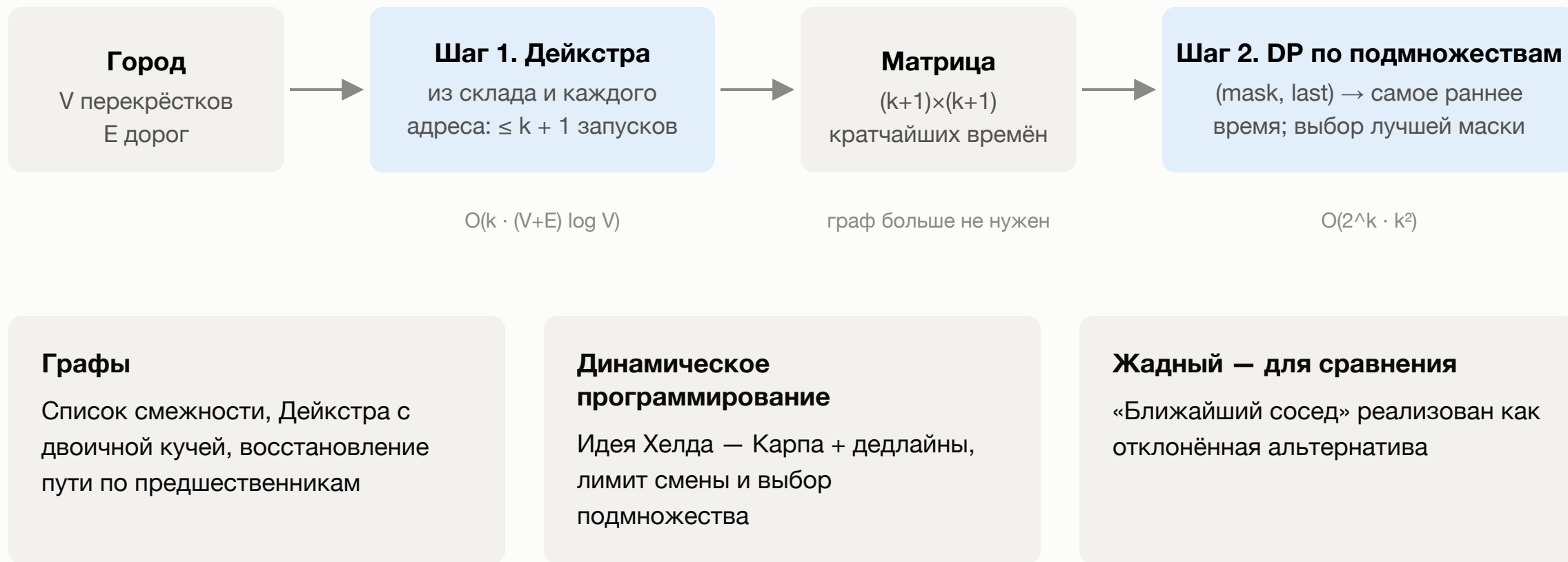
Максимум суммарной ценности; при равенстве — более раннее возвращение.

Ограничения

Прибытие к заказу $\leq$ его дедлайна
Возврат на склад $\leq T$

Нужно решить **сразу две задачи**: выбрать подмножество заказов (как в рюкзаке) и порядок объезда (как в задаче коммивояжёра). Задача NP-трудная.

Два шага: сжать граф города, затем точное DP



Дейкстра из $k + 1$ точек по списку смежности

- Ключевые точки — склад и адреса заказов; одинаковые адреса считаются один раз
- Куча `PriorityQueue` без `decrease-key`: устаревшие записи пропускаются при извлечении, записей $\leq E + 1$
- Для каждой точки хранится дерево предшественников — по нему восстанавливается путь по улицам
- Отрицательное время проезда отвергается при вводе: на нём Дейкстра некорректна

ВАРИАНТ	ПОЧЕМУ
выбрано Список смежности	Город разреженный: $E \approx 4V$. Память $O(V + E)$, Дейкстра идёт только по реальным дорогам
отклонено Матрица смежности	$O(V^2)$ памяти; каждый шаг просматривает строку из V пустых ячеек
выбрано $k + 1$ раз Дейкстра	$\sim 1,5 \cdot 10^6$ операций при $V = 1600$, $k = 20$; замер — 8 мс
отклонено Флойд — Уоршелл	$O(V^3) \approx 4 \cdot 10^9$ ради всех пар, а нужны $(k+1)^2$. Оставлен эталоном в тестах
отклонено BFS	Находит путь с минимумом дорог, а не минут

Состояние — какие заказы доставлены и где курьер

finish[mask, last] — самая ранняя минута, когда курьер закончил заказ **last**, доставив ровно множество **mask**, не нарушив дедлайнов и ещё успевая на склад

- **База:** $\text{finish}[\{j\}, j] = \text{dist}(\text{склад}, j) + \text{service}(j)$
- **Порядок:** маски по возрастанию — переход только добавляет бит
- **Ответ:** $\max \text{value}(\text{mask})$ по всем достигнутым состояниям; пустой маршрут допустим всегда
- **Путь:** массив $\text{previous}[\text{mask}, \text{last}] \rightarrow$ восстановление с конца

```
// DpRoutePlanner.cs — переход (mask, last) → next
long arrival = time + dist(last, next);
if (arrival > deadline[next]) return;

long done = arrival + service[next];

// отсечение: со склада уже не успеть —
// никакое продолжение не поможет
if (done + back[next] > shift) return;

int i = (mask | 1 << next) * k + next;
if (done < finish[i])
{
    finish[i] = (int)done;          // храним минимум
    previous[i] = last;
}
```

Достаточно хранить одно число — самое раннее время

1. **Будущее зависит только от состояния:** где курьер (last), что доставлено (mask) и который час
2. **Раньше — не хуже.** Все ограничения — *верхние* границы (дедлайн, конец смены). Продолжение позднего маршрута выполнимо и для раннего — моменты прибытия лишь сдвигаются раньше
3. **Ценность определяется маской** и не зависит от порядка внутри неё → поздний маршрут можно отбросить
4. **Отсечение безопасно:** dist — кратчайшие, путь до склада через другие адреса не короче прямого

Проверено экспериментом

- 1500 случайных задач: DP = полный перебор всех упорядоченных подмножеств
- Эталон независим: свой Флойд — Уоршелл вместо Дейкстры
- Мутация «хранить позднее время» → падают 3 теста

Граница довода: окна «не раньше» без права ждать или пробки без свойства FIFO ломают «раньше — не хуже»

DP, а не жадный: ближайший заказ «съедает» время



Смена 75 мин. Жадному на второй офис не хватает 5 минут ($80 > 75$).

Жадный от оптимума, %



На 1500 случайных задачах жадный никогда не лучше DP. Свойства жадного выбора (как каноничность монет в КТ5) у задачи нет.

Чем этот проект отличается от маршрута дрона (КТ7)

Сжатие графа вместо поиска по состояниям города

Можно было, как в КТ7, искать по состояниям (перекрёсток, mask) прямо в городе.

СОСТОЯНИЙ ПРИ $V = 1600$, $K = 20$

(перекрёсток, mask)	$V \cdot 2^K \approx 1,7$ млрд
(last, mask)	$k \cdot 2^K \approx 21$ млн

Шаг 1 убирает V из экспоненциальной части — в 80 раз меньше состояний. Между заказами курьер всегда едет кратчайшим путём: раньше — не хуже.

Время — значение, а не координата состояния

КТ7: заряд был частью ключа (станция, заряд) — «дешевле» и «больше заряда» конфликтуют, ни один путь не доминирует.

Здесь: «раньше» — единственный критерий, ценность задаётся маской. Конфликта нет → храним одно число.

Таблица в T раз меньше, чем $dp[mask][last][время]$: при смене 480 мин — в 480 раз.

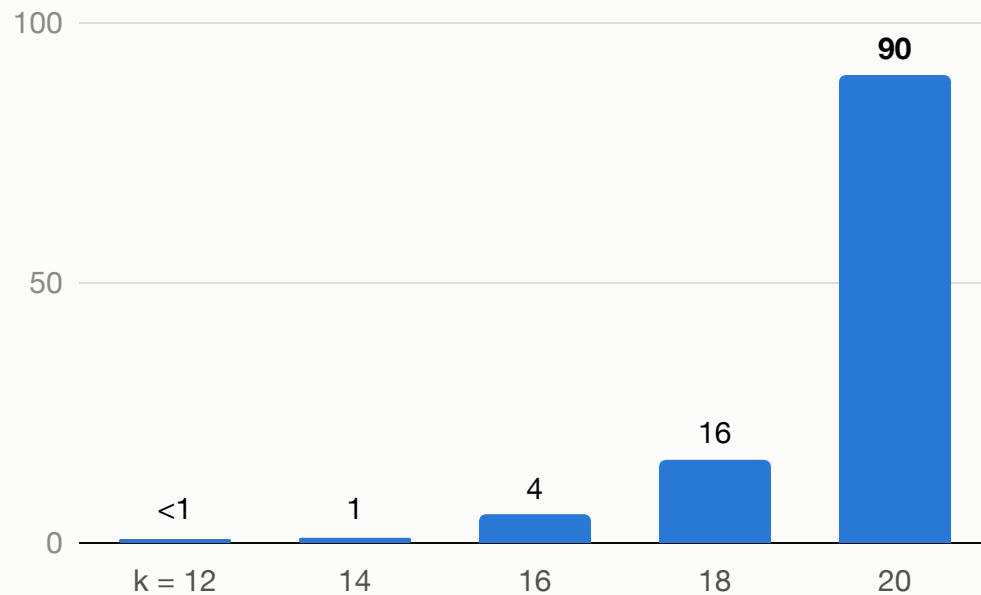
СЛОЖНОСТЬ

$O(k \cdot (V+E) \log V + 2^k \cdot k^2)$ — и упирается всё в память

ЧАСТЬ	ВРЕМЯ	ПАМЯТЬ
Дейкстры	$O(k \cdot (V+E) \log V)$	$O(k \cdot V)$
DP: циклы <code>mask × last × next</code>	$O(2^k \cdot k^2)$	$O(2^k \cdot k)$
Сборка плана	$O(k + \text{путь})$	то же

- Каждый новый заказ **удваивает** таблицу DP
- $k = 20 \rightarrow \sim 100$ МБ; $k = 25 \rightarrow \sim 4$ ГБ; $k = 30 \rightarrow \sim 160$ ГБ
- Поэтому $k > 20$ отклоняется явной ошибкой, а не падением по памяти
- Рекурсии в решении нет — стек от входа не растёт

Время DP, мс · решётка 40×40, Release



Дейкстры при этом — 6–8 мс независимо от k

Что ещё рассматривалось и почему нет

ПОДХОД	СЛОЖНОСТЬ	ПРИЧИНА ОТКАЗА
Полный перебор порядков	$O(e \cdot k! \cdot k)$	$k = 12 \rightarrow \sim 1,3 \cdot 10^9$ вариантов. DP схлопывает перестановки одного множества с одним концом. Оставлен эталоном в тестах
Жадный «ближайший сосед»	$O(k^2)$	Не оптимален: контрпримеры 01–03, до –50 % ценности
Жадный «ценность за минуту»	$O(k^2)$	Срочный заказ важнее выгодного по отношению; свойства жадного выбора нет
Рюкзак по бюджету времени	$O(k \cdot T)$	Время на заказ зависит от того, откуда едет курьер, — не аддитивно
Поиск по (перекрёсток, mask)	$O(V \cdot 2^k \cdot k \cdot \log)$	В V / k раз больше состояний
Флойд — Уоршелл · матрица смежности	$O(V^3) \cdot O(V^2)$	Ненужные пары; граф разреженный
Ветви и границы, ЦЛП, 2-opt, отжиг	экспонента / без гарантий	Нужны при $k \gg 20$; точные — сложны, эвристики не гарантируют оптимум

Ответ есть всегда: пустой маршрут допустим

Пограничные случаи каждый — тест

- Заказов нет → путь [Склад] , возврат в 0
- Один заказ ровно впритык / на минуту дольше смены
- Дедлайн ровно в момент прибытия → успел
- Смена 0 → только заказы на самом складе
- Ни один заказ не выполним → у каждого своя причина:
недостижим · нет пути обратно (одностороннее) · не успеть к дедлайну · не укладывается в смену
- Несколько заказов по одному адресу; повторный проезд перекрёстка
- Ровно 20 заказов — решается; 21 — явная ошибка

Некорректный ввод отклоняется явно

- **Отрицательное время проезда** — ломает инвариант Дейкстры; при отрицательном цикле кратчайшего пути нет
- Отрицательные смена, дедлайн, время передачи; ценность ≤ 0
- Повтор номера заказа, адрес или склад вне сети, битый JSON
- Циклы в графе — норма для города, обрабатываются без особой логики
- Суммы времени — в long ; RouteBuilder перепроверяет любой план

42 теста и 7 сценариев

42

теста xUnit, ~1 с

1500

задач сверено с полным перебором

- **Дейкстра** = Флойд — Уоршелл на 300 случайных графах
- **Каждый план** перепроверяется независимо: путь из реальных дорог, времена, дедлайны, смена
- **Жадный** всегда допустим и никогда не лучше DP
- **Мутационная проверка:** сломанный DP ловят 3 теста

```
# из папки project/  
dotnet test  
  
# основной сценарий: DP 19 против жадного 16  
dotnet run --project src/CourierRouting.Cli \  
  -- scenarios/01-city.json  
  
# ловушка для жадного и пограничный случай  
... -- scenarios/02-greedy-trap.json  
... -- scenarios/04-edge-unreachable.json  
  
# некорректный ввод: отрицательная дорога  
... -- scenarios/07-invalid-negative.json  
  
# рост времени с числом заказов  
dotnet run -c Release ... -- --bench
```

12 перекрёстков, 8 заказов, смена 2 часа

DP — ценность 19, возврат в 100 мин

17 **F** Школа дедлайн 25

24 **B** Больница дедлайн 60

36 **H** Парк

50 **C** Университет

59 **D** Стадион дедлайн 90

70 **E** Набережная

86 **G** Завод

Не доставлен А (Вокзал, дедлайн 30) — не помещается вместе с F

Жадный — ценность 16, возврат в 88 мин

12 **G** Завод ближайший, ценность 1

28 **E** Набережная

38 **D** Стадион

47 **C** Университет

60 **B** Больница впритык к дедлайну

72 **H** Парк

Поехал к ближайшему заводу и **не успел к срочному F**.
Смена использована не полностью (88 из 120 мин), а
срочный заказ уже потерян

Что сломается первым при изменении условий

ИЗМЕНЕНИЕ	ЧТО ПРОИЗОЙДЁТ
Заказов — сотни	Первой ломается память DP ($2^k \cdot k$). Дальше — кластеризация по районам + DP внутри, ветви и границы или эвристики
Город — миллионы перекрёстков	Растёт только шаг 1, линейно по k . Двухнаправленная Дейкстра, A^* , contraction hierarchies; $k + 1$ запусков параллелятся тривиально
Окна «не раньше»	С правом ждать — DP корректен: $arrival = \max(arrival, opens)$. Без права ждать доминирование ломается → время становится измерением, как заряд в KT7
Пробки (время зависит от момента)	Корректно, пока выполняется FIFO: выехав раньше, не приедешь позже
Вместимость сумки	Проверка по mask — размер состояния тот же
Несколько курьеров	Разбиение подмножеств между курьерами — $O(3^k)$; на практике эвристики
Распараллеливание	Узкое место — DP: параллелится только внутри слоя $ mask = m$, упор в пропускную способность памяти

Комбинация двух техник курса — и обоснование каждой

Графы

Список смежности и Дейкстра
сжимают город из V перекрёстков
до $k + 1$ точек

DP по подмножествам

Точный оптимум за $O(2^k \cdot k^2)$;
корректность доказана
доминированием и проверена
перебором

Жадный отклонён

Контрпримеры в тестах и демо:
теряет до половины ценности

Главная граница метода — память при $k > 20$. Она осознанная: до 20 заказов DP проще и точнее любых альтернатив, а превышение отклоняется явно

Спасибо за внимание! Вопросы?